

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: CSA0401

COURSE CODE: Operating Systems

COURSE OUTCOME COVERED

CO4: Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25

Annexure A

CODING CHALLENGE/HACKATHON

Code of Conduct:

I ___G. Yashmita (192511032)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G. Yashmita

Annexure A

CODING CHALLENGE

S.No	Coding Challenge Questions	Marks
1	Write a program to simulate the three file allocation methods—Contiguous, Linked, and Indexed. The program should accept inputs including the total disk size (number of blocks), the file size(number of blocks required),and the allocation method (1forContiguous, 2for Linked, 3 for Indexed). Based on the selected method, the program must allocate disk blocks accordingly, display the allocated block numbers, and handle fragmentation scenarios, particularly when contiguous allocation fails due to insufficient continuous space. The output should show the list of allocated blocks or an appropriate error message if allocation is not possible. Ensure that built-in memory allocation functions are not used, and assume that disk blocks are numbered starting from 0.	5
2	Develop a program to manage free disk space using a bitmap technique. The program should accept inputs including the total number of disk blocks, the initial bitmap (where 0 represents free and 1 represents allocated), and a request for allocation specifying the number of blocks required. The program must identify and allocate free blocks efficiently, update the bitmap after allocation, and also support deallocation of specified blocks with appropriate bitmap updates. The output should display the updated bitmap after allocation and deallocation operations. Ensure that the program efficiently searches for free blocks and properly handles Cases where sufficient space is not available.	5
3	Implement a program that simulates at least three disk scheduling algorithms: FCFS, SSTF, and SCAN. The program should take as input the disk request queue, the initial head position, and the disk size. It must compute the total seek time for each algorithm, display the order in which disk requests are serviced, and compare their performance. The output should include the seek sequence and total seek time for each algorithm, along with identification of the best-Performing algorithm based on minimum seek time.	5
4	Write a program to implement the C-SCAN and LOOK disk scheduling algorithms. The program should accept inputs including the request queue, the initial head position, and the direction of head movement (left or right). It must simulate both algorithms, calculate the total head movement required to service all requests, and compare their efficiency. The output should display the seek sequence, total head movement, and a conclusion indicating which algorithm performs better along with a brief justification. Optionally, the program may include a graphical visualization of head movement.	5
5	Design a program that integrates file allocation strategies with disk scheduling techniques to optimize system performance. The program should accept inputs such as the total number of disk blocks, file size (number of blocks required), disk request queue, initial head position, allocation method (Contiguous or Indexed), and disk scheduling method (FCFS or SSTF). It must allocate disk blocks based on the chosen allocation strategy, simulate disk scheduling to access the allocated blocks, calculate the total seek time, and compare performance across different combinations. The output should display allocated blocks, seek sequence, total seek time, and identify the best combination of allocation and scheduling methods. Ensure the program handles allocation failures, avoids duplicate block allocation, and maintains efficient scheduling.	5

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Question 1 — File Allocation Methods

Requirements covered

- Contiguous allocation
- Linked allocation
- Indexed allocation
- Total disk size
- File size
- Allocation method
- Block numbers start from 0
- Detects contiguous fragmentation
- Avoids duplicate allocation
- No dynamic memory allocation

Code:

```
#include <stdio.h>
```

```
#define MAX 1000
```

```
void displayBlocks(int blocks[], int n) {  
    int i;
```

```
    printf("Allocated blocks: ");  
    for (i = 0; i < n; i++) {  
        printf("%d", blocks[i]);  
        if (i != n - 1)  
            printf(" -> ");  
    }  
    printf("\n");  
}
```

```
void contiguous(int disk[], int n, int fileSize) {
```

```
    int start = -1;  
    int count = 0;  
    int i, j;
```

```
    for (i = 0; i < n; i++) {  
        if (disk[i] == 0) {  
            count++;  
  
            if (count == fileSize) {  
                start = i - fileSize + 1;  
                break;  
            }  
        } else {  
            count = 0;  
        }  
    }
```

```

    }

    if (start == -1) {
        printf("Contiguous allocation failed.\n");
        printf("Reason: Sufficient continuous free space is not available.\n");
        return;
    }

    printf("Contiguous allocation successful.\n");
    printf("Allocated blocks: ");

    for (i = start; i < start + fileSize; i++) {
        printf("%d ", i);
        disk[i] = 1;
    }

    printf("\n");
}

void linked(int disk[], int n, int fileSize) {
    int blocks[MAX];
    int count = 0;
    int i;

    for (i = 0; i < n && count < fileSize; i++) {
        if (disk[i] == 0) {
            blocks[count++] = i;
            disk[i] = 1;
        }
    }

    if (count < fileSize) {
        printf("Linked allocation failed.\n");
        printf("Reason: Not enough free blocks.\n");
        return;
    }

    printf("Linked allocation successful.\n");
    displayBlocks(blocks, fileSize);
}

void indexed(int disk[], int n, int fileSize) {
    int indexBlock = -1;
    int blocks[MAX];
    int count = 0;
    int i;

    /* Find index block */
    for (i = 0; i < n; i++) {
        if (disk[i] == 0) {

```

```

        indexBlock = i;
        disk[i] = 1;
        break;
    }
}

if (indexBlock == -1) {
    printf("Indexed allocation failed.\n");
    printf("Reason: No free block available for index block.\n");
    return;
}

/* Find data blocks */
for (i = 0; i < n && count < fileSize; i++) {
    if (disk[i] == 0) {
        blocks[count++] = i;
        disk[i] = 1;
    }
}

if (count < fileSize) {
    printf("Indexed allocation failed.\n");
    printf("Reason: Not enough free blocks.\n");
    disk[indexBlock] = 0;
    return;
}

printf("Indexed allocation successful.\n");
printf("Index block: %d\n", indexBlock);

displayBlocks(blocks, fileSize);
}

int main() {
    int disk[MAX];
    int n, fileSize, method;
    int i;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &n);

    printf("Enter initial disk status (0 = free, 1 = allocated):\n");

    for (i = 0; i < n; i++) {
        scanf("%d", &disk[i]);
    }

    printf("Enter file size in blocks: ");
    scanf("%d", &fileSize);

```

```

if (fileSize <= 0 || fileSize > n) {
    printf("Invalid file size.\n");
    return 0;
}

printf("\n1. Contiguous\n");
printf("2. Linked\n");
printf("3. Indexed\n");

printf("Enter allocation method: ");
scanf("%d", &method);

switch (method) {
    case 1:
        contiguous(disk, n, fileSize);
        break;

    case 2:
        linked(disk, n, fileSize);
        break;

    case 3:
        indexed(disk, n, fileSize);
        break;

    default:
        printf("Invalid allocation method.\n");
}

return 0;
}

```

```

main.c
1 #include <stdio.h>
2
3 #define MAX 1000
4
5 void displayBlocks(int blocks[], int n) {
6     int i;
7
8     printf("Allocated blocks: ");
9     for (i = 0; i < n; i++) {
10         printf("%d", blocks[i]);
11         if (i != n - 1)
12             printf(" -> ");
13     }
14     printf("\n");
15 }
16
17 void contiguous(int disk[], int n, int fileSize) {
18     int start = -1;
19     int count = 0;
20     int i, j;
21
22     for (i = 0; i < n; i++) {
23         if (disk[i] == 0) {
24             count++;
25
26             if (count == fileSize) {
27                 start = i - fileSize + 1;
28                 break;
29             }
30         } else {
31             count = 0;
32         }
33     }
34
35     if (start == -1) {
36         printf("Contiguous allocation failed.\n");
37         printf("Reason: Sufficient continuous free space is not available.\n");
38         return;
39     }
40
41     printf("Contiguous allocation successful.\n");
42     printf("Allocated blocks: ");
43 }

```

```
printf("Enter total number of disk blocks: ");
scanf("%d", &n);

printf("Enter initial disk status (0 = free, 1 = allocated):\n");
for (i = 0; i < n; i++) {
    scanf("%d", &disk[i]);
}

printf("Enter file size in blocks: ");
scanf("%d", &fileSize);

if (fileSize <= 0 || fileSize > n) {
    printf("Invalid file size.\n");
    return 0;
}

printf("\n1. Contiguous\n");
printf("2. Linked\n");
printf("3. Indexed\n");

printf("Enter allocation method: ");
scanf("%d", &method);

switch (method) {
    case 1:
        contiguous(disk, n, fileSize);
        break;
    case 2:
        linked(disk, n, fileSize);
        break;
    case 3:
        indexed(disk, n, fileSize);
        break;
    default:
        printf("Invalid allocation method.\n");
}

return 0;
}
```

input

```
Enter total number of disk blocks: 10
Enter initial disk status (0 = free, 1 = allocated):
1 0 0 1 0 0 0 1 0 0
Enter file size in blocks: 3

1. Contiguous
2. Linked
3. Indexed
Enter allocation method: 1
Contiguous allocation successful.
Allocated blocks: 4 5 6

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 2 — Bitmap Free Space Management

Requirements covered

- Bitmap
- 0 = free
- 1 = allocated
- Allocation
- Deallocation
- Updated bitmap
- Insufficient-space handling
- Efficient free-block search

Code:

```
#include <stdio.h>
```

```
#define MAX 1000
```

```

void displayBitmap(int bitmap[], int n) {
    int i;

    printf("Bitmap: ");

    for (i = 0; i < n; i++) {
        printf("%d ", bitmap[i]);
    }

    printf("\n");
}

int main() {
    int bitmap[MAX];
    int n;
    int request;
    int deallocateCount;
    int block;
    int i;
    int freeBlocks = 0;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &n);

    printf("Enter initial bitmap (0 = free, 1 = allocated):\n");

    for (i = 0; i < n; i++) {
        scanf("%d", &bitmap[i]);

        if (bitmap[i] == 0)
            freeBlocks++;
    }

    displayBitmap(bitmap, n);

    /* Allocation */
    printf("\nEnter number of blocks to allocate: ");
    scanf("%d", &request);

    if (request <= 0) {
        printf("Invalid allocation request.\n");
    }
    else if (freeBlocks < request) {
        printf("Allocation failed.\n");
        printf("Only %d free blocks are available.\n", freeBlocks);
    }
    else {
        int allocated = 0;

```

```

printf("Allocated blocks: ");

for (i = 0; i < n && allocated < request; i++) {
    if (bitmap[i] == 0) {
        bitmap[i] = 1;
        printf("%d ", i);
        allocated++;
    }
}

printf("\nAllocation successful.\n");
displayBitmap(bitmap, n);
}

/* Deallocation */
printf("\nEnter number of blocks to deallocate: ");
scanf("%d", &deallocateCount);

if (deallocateCount < 0) {
    printf("Invalid deallocation count.\n");
}
else {
    for (i = 0; i < deallocateCount; i++) {
        printf("Enter block number to deallocate: ");
        scanf("%d", &block);

        if (block < 0 || block >= n) {
            printf("Invalid block number.\n");
        }
        else if (bitmap[block] == 0) {
            printf("Block %d is already free.\n", block);
        }
        else {
            bitmap[block] = 0;
            printf("Block %d deallocated successfully.\n", block);
        }
    }

    printf("\nBitmap after deallocation:\n");
    displayBitmap(bitmap, n);
}

return 0;
}

```

```

1 #include <stdio.h>
2
3 #define MAX 1000
4
5 void displayBitmap(int bitmap[], int n) {
6     int i;
7     printf("Bitmap: ");
8     for (i = 0; i < n; i++) {
9         printf("%d ", bitmap[i]);
10    }
11    printf("\n");
12 }
13
14 int main() {
15     int bitmap[MAX];
16     int n;
17     int request;
18     int deallocateCount;
19     int block;
20     int i;
21     int freeBlocks = 0;
22
23     printf("Enter total number of disk blocks: ");
24     scanf("%d", &n);
25
26     printf("Enter initial bitmap (0 = free, 1 = allocated):\n");
27     for (i = 0; i < n; i++) {
28         scanf("%d", &bitmap[i]);
29         if (bitmap[i] == 0)
30             freeBlocks++;
31     }
32     displayBitmap(bitmap, n);
33
34     /* Allocation */
35     printf("\nEnter number of blocks to allocate: ");
36 }

```

```

    if (bitmap[i] == 0) {
        bitmap[i] = 1;
        printf("%d ", i);
        allocated++;
    }
}

printf("\nAllocation successful.\n");
displayBitmap(bitmap, n);

/* Deallocation */
printf("\nEnter number of blocks to deallocate: ");
scanf("%d", &deallocateCount);

if (deallocateCount < 0) {
    printf("Invalid deallocation count.\n");
} else {
    for (i = 0; i < deallocateCount; i++) {
        printf("Enter block number to deallocate: ");
        scanf("%d", &block);

        if (block < 0 || block >= n) {
            printf("Invalid block number.\n");
        } else if (bitmap[block] == 0) {
            printf("Block %d is already free.\n", block);
        } else {
            bitmap[block] = 0;
            printf("Block %d deallocated successfully.\n", block);
        }
    }

    printf("\nBitmap after deallocation:\n");
    displayBitmap(bitmap, n);
}

return 0;
}

```

```

input
Enter total number of disk blocks: 8
Enter initial bitmap (0 = free, 1 = allocated):
1 0 1 0 0 1 0 0
Bitmap: 1 0 1 0 0 1 0 0

Enter number of blocks to allocate: 3
Allocated blocks: 1 3 4
Allocation successful.
Bitmap: 1 1 1 1 1 1 0 0

Enter number of blocks to deallocate: 2
Enter block number to deallocate: 3
Block 3 deallocated successfully.
Enter block number to deallocate: 5
Block 5 deallocated successfully.

Bitmap after deallocation:
Bitmap: 1 1 1 0 1 0 0 0

...Program finished with exit code 0
Press ENTER to exit console.

```

Question 3 — FCFS, SSTF and SCAN

Requirements covered

- FCFS
- SSTF
- SCAN
- Request queue
- Initial head
- Disk size
- Seek sequence
- Total seek time
- Best algorithm

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 1000

void fcfs(int requests[], int n, int head) {
    int total = 0;
    int current = head;
    int i;

    printf("\n--- FCFS ---\n");
    printf("Seek sequence: %d", current);

    for (i = 0; i < n; i++) {
        total += abs(current - requests[i]);
        current = requests[i];
        printf(" -> %d", current);
    }

    printf("\nTotal seek time: %d\n", total);
}

int sstf(int requests[], int n, int head, int display) {
    int visited[MAX] = {0};
    int total = 0;
    int current = head;
    int i, j;
    int index, distance, minDistance;

    if (display) {
        printf("\n--- SSTF ---\n");
        printf("Seek sequence: %d", current);
    }
}
```

```

    }

    for (i = 0; i < n; i++) {
        index = -1;
        minDistance = 1000000;

        for (j = 0; j < n; j++) {
            if (!visited[j]) {
                distance = abs(current - requests[j]);

                if (distance < minDistance) {
                    minDistance = distance;
                    index = j;
                }
            }
        }

        visited[index] = 1;
        total += minDistance;
        current = requests[index];

        if (display)
            printf(" -> %d", current);
    }

    if (display)
        printf("\nTotal seek time: %d\n", total);

    return total;
}

void scan(int requests[], int n, int head, int diskSize) {
    int left[MAX], right[MAX];
    int leftCount = 0, rightCount = 0;
    int total = 0;
    int current = head;
    int i, j, temp;

    for (i = 0; i < n; i++) {
        if (requests[i] < head)
            left[leftCount++] = requests[i];
        else
            right[rightCount++] = requests[i];
    }

    /* Sort left side */
    for (i = 0; i < leftCount - 1; i++) {
        for (j = i + 1; j < leftCount; j++) {
            if (left[i] > left[j]) {
                temp = left[i];

```

```

        left[i] = left[j];
        left[j] = temp;
    }
}

/* Sort right side */
for (i = 0; i < rightCount - 1; i++) {
    for (j = i + 1; j < rightCount; j++) {
        if (right[i] > right[j]) {
            temp = right[i];
            right[i] = right[j];
            right[j] = temp;
        }
    }
}

printf("\n--- SCAN ---\n");
printf("Seek sequence: %d", current);

/* Move right */
for (i = 0; i < rightCount; i++) {
    total += abs(current - right[i]);
    current = right[i];
    printf("-> %d", current);
}

/* Move to disk end */
if (current != diskSize - 1) {
    total += abs(current - (diskSize - 1));
    current = diskSize - 1;
    printf("-> %d", current);
}

/* Reverse direction */
for (i = leftCount - 1; i >= 0; i--) {
    total += abs(current - left[i]);
    current = left[i];
    printf("-> %d", current);
}

printf("\nTotal seek time: %d\n", total);
}

int main() {
    int requests[MAX];
    int n, head, diskSize;
    int i;

    printf("Enter number of disk requests: ");

```

```

scanf("%d", &n);

printf("Enter request queue:\n");

for (i = 0; i < n; i++)
    scanf("%d", &requests[i]);

printf("Enter initial head position: ");
scanf("%d", &head);

printf("Enter disk size: ");
scanf("%d", &diskSize);

if (head < 0 || head >= diskSize) {
    printf("Invalid head position.\n");
    return 0;
}

for (i = 0; i < n; i++) {
    if (requests[i] < 0 || requests[i] >= diskSize) {
        printf("Invalid request: %d\n", requests[i]);
        return 0;
    }
}

fcfs(requests, n, head);

int fcfsTotal = 0;
int current = head;

for (i = 0; i < n; i++) {
    fcfsTotal += abs(current - requests[i]);
    current = requests[i];
}

int sstfTotal = sstf(requests, n, head, 1);

/* SCAN calculation separately */
int left[MAX], right[MAX];
int leftCount = 0, rightCount = 0;
int scanTotal = 0;
int temp;

for (i = 0; i < n; i++) {
    if (requests[i] < head)
        left[leftCount++] = requests[i];
    else
        right[rightCount++] = requests[i];
}

```

```

int j;

for (i = 0; i < leftCount - 1; i++) {
    for (j = i + 1; j < leftCount; j++) {
        if (left[i] > left[j]) {
            temp = left[i];
            left[i] = left[j];
            left[j] = temp;
        }
    }
}

for (i = 0; i < rightCount - 1; i++) {
    for (j = i + 1; j < rightCount; j++) {
        if (right[i] > right[j]) {
            temp = right[i];
            right[i] = right[j];
            right[j] = temp;
        }
    }
}

current = head;

for (i = 0; i < rightCount; i++) {
    scanTotal += abs(current - right[i]);
    current = right[i];
}

scanTotal += abs(current - (diskSize - 1));
current = diskSize - 1;

for (i = leftCount - 1; i >= 0; i--) {
    scanTotal += abs(current - left[i]);
    current = left[i];
}

scan(requests, n, head, diskSize);

printf("\n--- Performance Comparison ---\n");
printf("FCFS Total Seek Time : %d\n", fcfsTotal);
printf("SSTF Total Seek Time : %d\n", sstfTotal);
printf("SCAN Total Seek Time : %d\n", scanTotal);

if (fcfsTotal <= sstfTotal && fcfsTotal <= scanTotal)
    printf("Best performing algorithm: FCFS\n");
else if (sstfTotal <= fcfsTotal && sstfTotal <= scanTotal)
    printf("Best performing algorithm: SSTF\n");
else
    printf("Best performing algorithm: SCAN\n");

```

return 0;

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 1000
5
6 void fcfs(int requests[], int n, int head) {
7     int total = 0;
8     int current = head;
9     int i;
10
11     printf("\n--- FCFS ---\n");
12     printf("Seek sequence: %d", current);
13
14     for (i = 0; i < n; i++) {
15         total += abs(current - requests[i]);
16         current = requests[i];
17         printf(" -> %d", current);
18     }
19
20     printf("\nTotal seek time: %d\n", total);
21 }
22
23 int sstf(int requests[], int n, int head, int display) {
24     int visited[MAX] = {0};
25     int total = 0;
26     int current = head;
27     int i, j;
28     int index, distance, minDistance;
29
30     if (display) {
31         printf("\n--- SSTF ---\n");
32         printf("Seek sequence: %d", current);
33     }
34
35     for (i = 0; i < n; i++) {
36         index = -1;
37         minDistance = 1000000;
38
39         for (j = 0; j < n; j++) {
40             if (!visited[j]) {
41                 distance = abs(current - requests[j]);
42
43                 if (distance < minDistance) {
44                     minDistance = distance;
45                     index = j;
46                 }
47             }
48         }
49
50         if (index != -1) {
51             total += minDistance;
52             current = requests[index];
53             visited[index] = 1;
54         }
55     }
56
57     if (display) {
58         printf("\nTotal seek time: %d\n", total);
59     }
60 }
61
62 int scan(int requests[], int n, int head, int diskSize) {
63     int leftCount = 0;
64     int rightCount = 0;
65     int left[n];
66     int right[n];
67     int i, j;
68     int current = head;
69     int scanTotal = 0;
70
71     for (i = 0; i < n; i++) {
72         if (requests[i] < current) {
73             left[leftCount++] = requests[i];
74         }
75         if (requests[i] > current) {
76             right[rightCount++] = requests[i];
77         }
78     }
79
80     for (i = 0; i < rightCount; i++) {
81         scanTotal += abs(current - right[i]);
82         current = right[i];
83     }
84
85     scanTotal += abs(current - (diskSize - 1));
86     current = diskSize - 1;
87
88     for (i = leftCount - 1; i >= 0; i--) {
89         scanTotal += abs(current - left[i]);
90         current = left[i];
91     }
92
93     return scanTotal;
94 }
95
96 int main() {
97     int n, requests[MAX], head, diskSize;
98
99     printf("Enter number of disk requests: ");
100     scanf("%d", &n);
101
102     printf("Enter request queue: ");
103     for (int i = 0; i < n; i++) {
104         scanf("%d", &requests[i]);
105     }
106
107     printf("Enter initial head position: ");
108     scanf("%d", &head);
109
110     printf("Enter disk size: ");
111     scanf("%d", &diskSize);
112
113     fcfs(requests, n, head);
114     sstf(requests, n, head, 1);
115     scan(requests, n, head, diskSize);
116
117     printf("\n--- Performance Comparison ---\n");
118     printf("FCFS Total Seek Time : %d\n", fcfsTotal);
119     printf("SSTF Total Seek Time : %d\n", sstfTotal);
120     printf("SCAN Total Seek Time : %d\n", scanTotal);
121
122     if (fcfsTotal <= sstfTotal && fcfsTotal <= scanTotal) {
123         printf("Best performing algorithm: FCFS\n");
124     }
125     else if (sstfTotal <= fcfsTotal && sstfTotal <= scanTotal) {
126         printf("Best performing algorithm: SSTF\n");
127     }
128     else {
129         printf("Best performing algorithm: SCAN\n");
130     }
131
132     return 0;
133 }
```

```
Enter number of disk requests: 8
Enter request queue:
35 63 57 134 156 13 86 89
Enter initial head position: 53
Enter disk size: 200

--- FCFS ---
Seek sequence: 53 -> 35 -> 63 -> 57 -> 134 -> 156 -> 13 -> 86 -> 89
Total seek time: 370

--- SSTF ---
Seek sequence: 53 -> 57 -> 63 -> 86 -> 89 -> 134 -> 156 -> 35 -> 13
Total seek time: 246

--- SCAN ---
Seek sequence: 53 -> 57 -> 63 -> 86 -> 89 -> 134 -> 156 -> 199 -> 35 -> 13
Total seek time: 332

--- Performance Comparison ---
FCFS Total Seek Time : 370
SSTF Total Seek Time : 246
SCAN Total Seek Time : 332
Best performing algorithm: SSTF

...Program finished with exit code 0
Press ENTER to exit console.[]
```

Question 4 — C-SCAN and LOOK

This question explicitly asks for **direction**, so this program accepts:

L = Left

R = Right

Requirements covered

- C-SCAN
- LOOK
- Initial head
- Request queue
- Direction
- Total head movement
- Seek sequence
- Comparison
- Conclusion

Code:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

int main() {
    int a[MAX],n,h,d,i,j,x,cur,cscan=0,look=0;
    char dir;

    printf("Requests: ");
    scanf("%d",&n);

    for(i=0;i<n;i++) scanf("%d",&a[i]);

    printf("Initial head: ");
    scanf("%d",&h);

    printf("Disk size: ");
    scanf("%d",&d);

    printf("Direction (L/R): ");
    scanf(" %c",&dir);

    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                x=a[i]; a[i]=a[j]; a[j]=x;
            }
}
```

```

/* C-SCAN */
cur=h;
printf("\nC-SCAN: %d",cur);

if(dir=='R' || dir=='r') {

    for(i=0;i<n;i++)
        if(a[i]>=h) {
            cscan+=abs(cur-a[i]);
            cur=a[i];
            printf(" -> %d",cur);
        }

    cscan+=abs(cur-(d-1));
    cur=d-1;
    printf(" -> %d",cur);

    cscan+=d-1;
    cur=0;
    printf(" -> %d",cur);

    for(i=0;i<n;i++)
        if(a[i]<h) {
            cscan+=abs(cur-a[i]);
            cur=a[i];
            printf(" -> %d",cur);
        }
}
else {

    for(i=n-1;i>=0;i--)
        if(a[i]<=h) {
            cscan+=abs(cur-a[i]);
            cur=a[i];
            printf(" -> %d",cur);
        }

    cscan+=cur;
    cur=0;
    printf(" -> %d",cur);

    cscan+=d-1;
    cur=d-1;
    printf(" -> %d",cur);

    for(i=n-1;i>=0;i--)
        if(a[i]>h) {
            cscan+=abs(cur-a[i]);
            cur=a[i];
            printf(" -> %d",cur);
        }
}

```

```

    }
}

printf("\nTotal = %d\n",cscan);

/* LOOK */
cur=h;
printf("\nLOOK: %d",cur);

if(dir=='R' || dir=='r') {

    for(i=0;i<n;i++)
        if(a[i]>=h) {
            look+=abs(cur-a[i]);
            cur=a[i];
            printf(" -> %d",cur);
        }

    for(i=n-1;i>=0;i--)
        if(a[i]<h) {
            look+=abs(cur-a[i]);
            cur=a[i];
            printf(" -> %d",cur);
        }
}
else {

    for(i=n-1;i>=0;i--)
        if(a[i]<=h) {
            look+=abs(cur-a[i]);
            cur=a[i];
            printf(" -> %d",cur);
        }

    for(i=0;i<n;i++)
        if(a[i]>h) {
            look+=abs(cur-a[i]);
            cur=a[i];
            printf(" -> %d",cur);
        }
}

printf("\nTotal = %d\n",look);

printf("\nBetter: %s",
       cscan<look ? "C-SCAN" :
       look<cscan ? "LOOK" : "Both");

return 0;
}

```

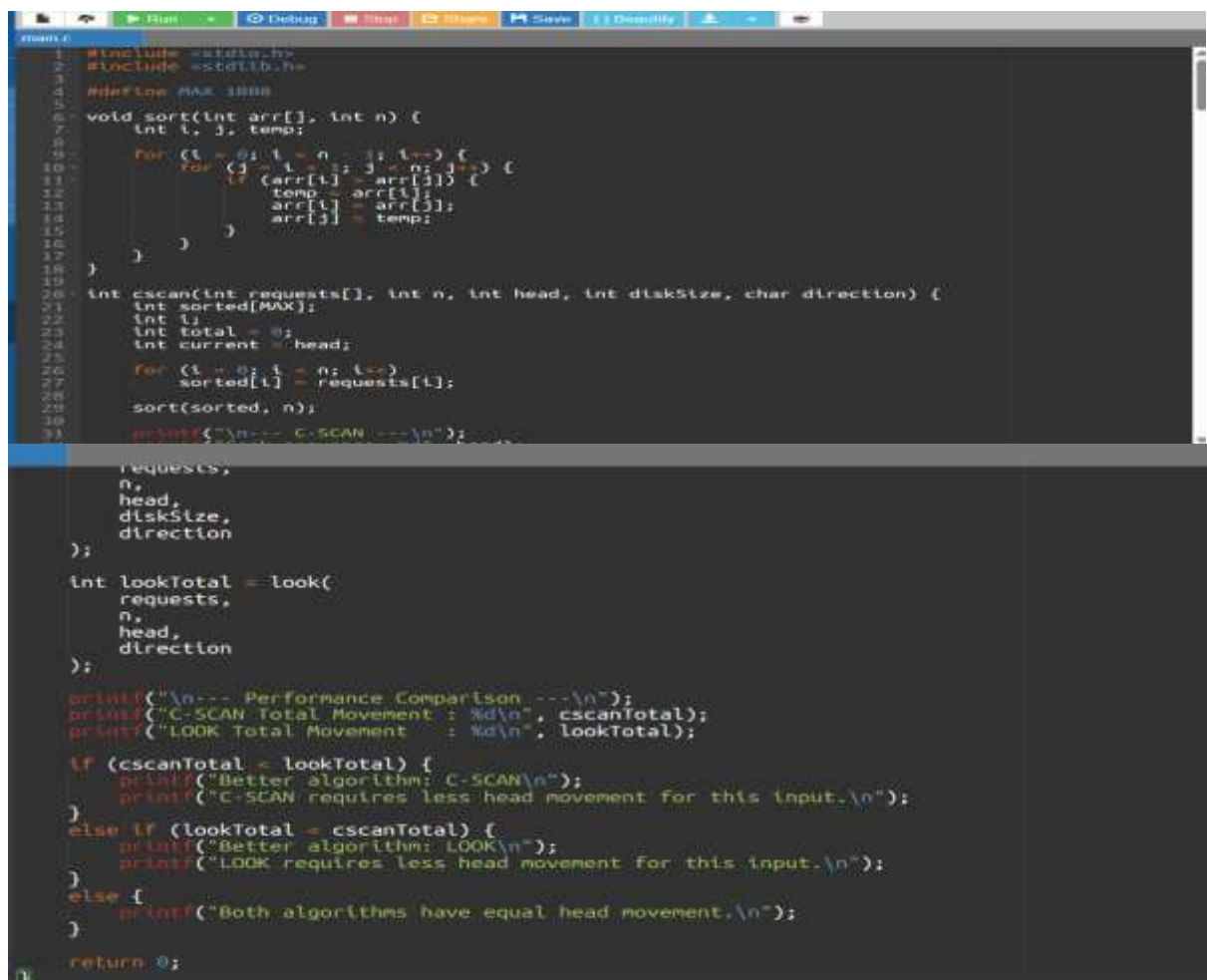
Important difference to explain

C-SCAN:

- Goes in one direction.
- Goes all the way to the disk boundary.
- Jumps to the opposite end.
- Continues servicing requests.

LOOK:

- Goes only as far as the last request.
- Does **not** travel unnecessarily to the disk boundary.
- Reverses direction when there are no more requests in that direction.



```
1 #include <iostream.h>
2 #include <stdlib.h>
3
4 #define MAX 10000
5
6 void sort(int arr[], int n) {
7     int i, j, temp;
8
9     for (i = 0; i < n - 1; i++) {
10         for (j = i + 1; j < n; j++) {
11             if (arr[i] > arr[j]) {
12                 temp = arr[i];
13                 arr[i] = arr[j];
14                 arr[j] = temp;
15             }
16         }
17     }
18 }
19
20 int cscan(int requests[], int n, int head, int diskSize, char direction) {
21     int sorted[MAX];
22     int i;
23     int total = 0;
24     int current = head;
25
26     for (i = 0; i < n; i++)
27         sorted[i] = requests[i];
28
29     sort(sorted, n);
30
31     printf("\n--- C-SCAN ---\n");
32
33     requests,
34     n,
35     head,
36     diskSize,
37     direction
38 );
39
40 int lookTotal = look(
41     requests,
42     n,
43     head,
44     direction
45 );
46
47 printf("\n--- Performance Comparison ---\n");
48 printf("C-SCAN Total Movement : %d\n", cscanTotal);
49 printf("LOOK Total Movement : %d\n", lookTotal);
50
51 if (cscanTotal < lookTotal) {
52     printf("Better algorithm: C-SCAN\n");
53     printf("C-SCAN requires less head movement for this input.\n");
54 }
55 else if (lookTotal < cscanTotal) {
56     printf("Better algorithm: LOOK\n");
57     printf("LOOK requires less head movement for this input.\n");
58 }
59 else {
60     printf("Both algorithms have equal head movement.\n");
61 }
62
63 return 0;
64 }
```

```
input
Enter number of disk requests: 8
Enter request queue:
98 183 35 125 13 124 65 68
Enter initial head position: 53
Enter disk size: 200
Enter direction (L - Left, R - Right): R

--- C-SCAN ---
Seek sequence: 53 -> 65 -> 68 -> 98 -> 124 -> 125 -> 183 -> 199 -> 0 -> 13 -> 35
Total head movement: 380

--- LOOK ---
Seek sequence: 53 -> 65 -> 68 -> 98 -> 124 -> 125 -> 183 -> 35 -> 13
Total head movement: 300

--- Performance Comparison ---
C-SCAN Total Movement : 380
LOOK Total Movement   : 300
Better algorithm: LOOK
LOOK requires less head movement for this input.

...Program finished with exit code 0
Press ENTER to exit console.
```

Question 5 — Integrated File Allocation + Disk Scheduling

This is the most comprehensive question and is worth preparing carefully.

Requirements covered

- Total disk blocks
- File size
- Request queue
- Initial head
- Contiguous allocation
- Indexed allocation
- FCFS
- SSTF
- Allocated blocks
- Seek sequence
- Total seek time
- Compare combinations
- Best combination
- Allocation failure
- Duplicate avoidance

Code:

```
#include <stdio.h>

#include <stdlib.h>

#define MAX 100
```

```

int cont(int d[],int n,int f,int b[]) {

    int i,c=0,j;

    for(i=0;i<n;i++) {

        if(d[i]==0)c++;

        else c=0;

        if(c==f) {

            for(j=0;j<f;j++) {

                b[j]=i-f+1+j;

                d[b[j]]=1;

            }

            return 1;

        }

    }

    return 0;

}

int indexAlloc(int d[],int n,int f,int b[]) {

    int i,c=0,idx=-1;

    for(i=0;i<n;i++)

        if(d[i]==0) {

            idx=i;

            d[i]=1;

            break;

        }

    if(idx==-1)return 0;

    for(i=0;i<n&& c<f;i++)

```

```

        if(d[i]==0) {

            b[c++]=i;

            d[i]=1;

        }

        if(c<f) {

            d[idx]=0;

            return 0;

        }

        printf("Index block = %d\n",idx);

        return 1;

    }

int fcfs(int b[],int n,int h) {

    int i,t=0;

    for(i=0;i<n;i++) {

        t+=abs(h-b[i]);

        h=b[i];

    }

    return t;

}

int sstf(int b[],int n,int h) {

    int v[MAX]={0},i,j,x,t=0;

    for(i=0;i<n;i++) {

        x=-1;

        for(j=0;j<n;j++)

            if(!v[j] &&

```

```

        (x== -1 || abs(h-b[j])<abs(h-b[x])))

        x=j;

        v[x]=1;

        t+=abs(h-b[x]);

        h=b[x];

    }

    return t;

}

void show(int b[],int n) {

    int i;

    printf("Blocks: ");

    for(i=0;i<n;i++)printf("%d ",b[i]);

    printf("\n");

}

int main() {

    int n,f,q,h,a[MAX],i;

    int b[MAX],d[MAX];

    int c1,c2,i1,i2;

    printf("Disk size: ");

    scanf("%d",&n);

    printf("File size: ");

    scanf("%d",&f);

    printf("Number of requests: ");

    scanf("%d",&q);

    printf("Request queue: ");

```

```

for(i=0;i<q;i++)scanf("%d",&a[i]);

printf("Initial head: ");

scanf("%d",&h);

/* Contiguous */

for(i=0;i<n;i++)d[i]=0;

printf("\nCONTIGUOUS\n");

if(cont(d,n,f,b)) {

    show(b,f);

    c1=fcfs(b,f,h);

    c2=sstf(b,f,h);

    printf("FCFS = %d\n",c1);

    printf("SSTF = %d\n",c2);

}

else {

    printf("Contiguous allocation failed.\n");

    c1=c2=999999;

}

/* Indexed */

for(i=0;i<n;i++)d[i]=0;

printf("\nINDEXED\n");

if(indexAlloc(d,n,f,b)) {

    show(b,f);

    i1=fcfs(b,f,h);

    i2=sstf(b,f,h);

    printf("FCFS = %d\n",i1);

```

```
    printf("SSTF = %d\n",i2);
}
else {
    printf("Indexed allocation failed.\n");
    i1=i2=9999999;
}
printf("\nBEST COMBINATION\n");
if(c1<=c2 && c1<=i1 && c1<=i2)
    printf("Contiguous + FCFS = %d",c1);
else if(c2<=c1 && c2<=i1 && c2<=i2)
    printf("Contiguous + SSTF = %d",c2);
else if(i1<=c1 && i1<=c2 && i1<=i2)
    printf("Indexed + FCFS = %d",i1);
else
    printf("Indexed + SSTF = %d",i2);
return 0;
}
```

```
main.c Run Output
130
131 if(indexAlloc(d,n,f,b)) {
132     show(b,f);
133
134     it=fcs(b,f,h);
135     i2=astf(b,f,h);
136
137     printf("FCFS = %d\n",i1);
138     printf("SSTF = %d\n",i2);
139 }
140 else {
141     printf("Indexed allocation failed.\n");
142     i1=i2=999999;
143 }
144
145 printf("\nBEST COMBINATION\n");
146
147 if(c1<=c2 && c1<=i1 && c1<=i2)
148     printf("Contiguous + FCFS = %d",c1);
149 else if(c2<=c1 && c2<=i1 && c2<=i2)
150     printf("Contiguous + SSTF = %d",c2);
151 else if(i1<=c1 && i1<=c2 && i1<=i2)
152     printf("Indexed + FCFS = %d",i1);
153 else
154     printf("Indexed + SSTF = %d",i2);
155
Disk size: 20
File size: 5
Number of requests: 8
Request queue: 5 12 7 18 2 15 9 3
Initial head: 10

CONTIGUOUS
Blocks: 0 1 2 3 4
FCFS = 14
SSTF = 10

INDEXED
Index block = 0
Blocks: 1 2 3 4 5
FCFS = 13
SSTF = 9

BEST COMBINATION
Indexed + SSTF = 9

--- Code Execution Successful ---
```